BÀI TẬP LẬP TRÌNH NLP

Với thư viện NLTK, WORDNET

Bài 1. Tra cứu thông tin từ trong WordNet

Viết chương trình Python sử dụng NLTK để:

a. Liệt kê tất cả các synset của từ: "bank"

b. Với mỗi synset, in ra:

+ Tên synset

+ Định nghĩa (definition)

+ Ví dụ sử dụng (examples)

Bài 2. Phân tích cấu trúc quan hệ WordNet

Viết chương trình:

a. Lấy synset "car.n.01"

b. In ra danh sách:

+ Hypernyms (quan hệ bao hàm)
+ Hyponyms (quan hệ phân nhánh)
+ Meronyms (quan hệ bộ phận)
+ Holonyms (quan hệ toàn thể)

Bài 3. Tìm nghĩa gần đúng của từ trong câu

Cho câu sau: “The crane is standing near the river.”

Hãy viết chương trình:

a. Trích xuất từ “crane”

b. Dùng WordNet liệt kê tất cả nghĩa có thể.

c. Dựa vào câu, dự đoán nghĩa phù hợp nhất (ví dụ: animal vs machine).

Bài 4. Độ tương đồng bằng Path Similarity

Viết chương trình tính path similarity giữa các cặp từ sau: car - automobile, dog - cat, dog - wolf, tree – car.

In ra kết quả và nhận xét giá trị similarity.

Bài 5. Độ tương đồng bằng độ đo Wu-Palmer Similarity

(ví dụ: synset1.wup_similarity(synset2))

So sánh kết quả giữa path similarity và wup_similarity.

Bài 6. Tính độ tương đồng từ vựng dựa trên WordNet

Viết hàm:

def wordnet_similarity(word1, word2):

return the best similarity between any synset pairs

Hàm wordnet_similarity phải:

a. Lấy tất cả synset cho từng từ.

b. Tính WUP similarity cho mọi cặp synset.

c. Trả về giá trị similarity cao nhất.

Test với các cặp từ: *ship – boat, good -bad, food – fruit, computer – laptop*.

Bài 7. Xây dựng ứng dụng phân loại cảm xúc đơn giản dùng Word Similarity

Cho tập từ điển:

Positive = [“*good*”, “*excellent*”, “*happy*”, “*awesome*”]

Negative = [“*bad*”, “*terrible*”, “*sad*”, “*awful*”]

Viết chương trình:

a. Nhập vào một từ tiếng Anh bất kỳ.

b. Tính độ tương đồng với mỗi từ trong tập Positive và Negative.

c. Gán nhãn cho từ dựa theo nhóm có similarity cao hơn.

Bài 8. Phân cụm từ vựng bằng độ đo tương đồng

Cho danh sách từ:

[“*cat*”, “*dog*”, “*wolf*”, “*lion*”, “*car*”, “*truck*”, “*bus*”, “*airplane*”]

Yêu cầu:

a. Tính ma trận similarity giữa các từ bằng WUP similarity.

b. Áp dụng thuật toán phân cụm K-means.

c. In ra kết quả (các cụm động vật và phương tiện có tách được không?).

Bài 9. Tính semantic similarity cho câu (WordNet-based Sentence Similarity)

Viết hàm:

def sentence_similarity(sent1, sent2):

average max-similarity between words from the two sentences

Kiểm tra với các câu:

a. "*A dog is running in the park*."

b. "*A wolf is moving in the forest*."

c. "*A car is parked beside the road*."

Nhận xét kết quả.

Bài 10. Khử nhập nhằng ngữ nghĩa bằng thuật toán Lesk (Word Sense Disambiguation)

Sử dụng thuật toán Lesk trong NLTK (*from nltk.wsd import lesk*)

Viết chương trình chọn nghĩa phù hợp cho từ "bass" trong câu: "*I can hear the bass in this music*.", "*He caught a huge bass in the lake*."

Bài 11. Tìm từ similar bằng Word2Vec

Sử dụng mô hình Word2vecđã huấn luyện:

a. Viết chương trình in ra top-5 từ tương tự nhất của: "dog", "cat".

b. Nhận xét kết quả.

Bài 12. Tính độ tương đồng giữa từ bằng cosine similarity

Viết hàm:

def cosine_sim(word1, word2, model):

return cosine similarity

a. Dùng mô hình Word2Vec đã huấn luyện để tính similarity giữa các cặp từ: *"dog" – "cat", "dog" – "car", "king" – "queen".*

b. So sánh với similarity WordNet trước đó.